

ZFS Cheat Sheet

Pool your disks, carve the pool into filesystems, and take instant snapshots you can copy elsewhere. Needs sudo.

Pools vs datasets

disks	vdevs	pool	datasets (each one is a filesystem)
ata-A	mirror-0	← a mirror survives 1 disk dying; lose a whole vdev = lose the pool	
ata-B		← tank	mounted at /tank/home
ata-C	mirror-1	← tank/backups	mounted at /tank/backups
ata-D		← tank/vm	a zvol (block device): /dev/zvol/tank/vm

Building and looking after a pool

```
zpool create -o ashift=12 tank mirror /dev/disk/by-id/ata-A /dev/disk/by-id/ata-B ← by-id names don't change
zpool create -o ashift=12 tank raidz2 /dev/disk/by-id/ata-{A,B,C,D,E,F} ← raidz1 / raidz2 = survive 1 / 2 failures
zpool scrub tank zpool status -x zpool clear tank ← verify all data · show only problems · reset error counts
zpool replace tank ata-B /dev/disk/by-id/ata-NEW ← swap a failed disk; status shows the resilver
zpool export tank zpool import zpool import tank ← detach · list importable · bring it in
arc_summary ← RAM cache (ARC) stats; ZFS using lots of RAM is normal, it shrinks on demand
```

Reading zpool status

```
pool: tank
state: DEGRADED ← ONLINE = fine · DEGRADED = works, redundancy lost · FAULTED = pool down
scan: scrub repaired 0B in 03:12:44 with 0 errors on Sun Sep 13 03:36:45 2026 ← last scrub
config:
  NAME      STATE      READ WRITE CKSUM
  tank      DEGRADED   0     0     0
    mirror-0 DEGRADED   0     0     0
      ata-A  ONLINE    0     0     0
      ata-B  UNAVAIL    0     0     0 ← missing or dead: zpool replace it
    mirror-1 ONLINE     0     0     0
      ata-C  ONLINE    0     0    14 ← CKSUM: bad data found (fixed from ata-D); check cable/disk
      ata-D  ONLINE    0     0     0
errors: No known data errors ← otherwise lists damaged files
```

Reading zpool list and zfs list

NAME	SIZE	ALLOC	FREE	FRAG	CAP	HEALTH	
tank	10.9T	6.12T	4.78T	12%	56%	ONLINE	← zpool list -o name,size,alloc,free,frag,cap,health: raw space
							← keep CAP under ~80%; FRAG = free-space fragmentation
NAME		USED	AVAIL	REFER	MOUNTPOINT		← zfs list: usable space, after redundancy
tank		4.06T	3.10T	128K	/tank		← USED includes children + snapshots
tank/backups		3.20T	3.10T	2.71T	/tank/backups		← REFER = data you see now; the rest is held by snapshots
tank/home		860G	164G	812G	/tank/home		← AVAIL is lower: a 1T quota is set

Datasets and properties

Command	Does	Property	Means
zfs create tank/home	New dataset, mounts itself	compression=lz4	Fast, nearly free: use it
zfs set atime=off tank	Set; children inherit it	recordsize=1M	Big files (media, backups)
zfs get compressratio tank	Read a property	quota=1T	The most it may use
zfs get -s local all tank	Only what you changed	reservation=100G	Space guaranteed to it
zfs inherit atime tank	Back to the parent's value	mountpoint=/srv/x	Where it mounts

Snapshots and backups

```
zfs snapshot -r tank/home@2026-10-03 zfs list -t snapshot ← instant read-only copy (-r = children too) · list them
ls /tank/home/.zfs/snapshot/2026-10-03/ ← hidden dir: copy single files back
zfs rollback tank/home@2026-10-03 ← undo everything since (-r also deletes newer snaps)
zfs destroy tank/home@2026-09-01 ← delete one snapshot (-nv = dry run)
zfs destroy -r tank/home ← CAREFUL: deletes the dataset and ALL its snapshots, no prompt
zfs send tank/home@mon | ssh backup.example.com zfs receive pool2/home ← first full copy
zfs send -i @mon tank/home@tue | ssh backup.example.com zfs receive pool2/home ← then changes only
```

Remember: snapshots aren't backups until you **zfs send** them somewhere else. Scrub monthly.